Beslutsunderlag – ML-analys

California Housing Dataset | Spår A – Regression | Jonas Lindström

1. Mål & valt spår

Kunden är en fastighetsaktör som vill kunna göra grova värderingar av bostadsområden i Kalifornien för att planera investeringar och resurser. Målet är att kunna uppskatta **median_house_value** baserat på ett områdes egenskaper – utan att behöva känna till det faktiska priset i förväg.

Spår A – Regression valdes eftersom uppdraget är att förutsäga ett kontinuerligt värde (priset i dollar). Regression är rätt verktyg när svaret är ett tal på en skala, till skillnad från klassificering där man placerar i kategorier.

2. Kort data-insikt (EDA)

Datasetet innehåller 20 640 områden med 10 variabler. Variabeln *total_bedrooms* saknar 207 värden (~1 %) – hanterat med medianimputation i pipeline.

- **Pristaket vid 500 000 dollar:** Histogrammet visar att median_house_value är avklippt vid 500 000. Det innebär att modellen troligen underskattar värdet för de dyraste områdena.
- **Geografisk koncentration:** Scatter-plottet (longitud/latitud, färgat efter pris) visar att dyra områden klustrar kring kuststäderna – San Francisco, Los Angeles och San Diego. Inlandsområden har generellt lägre priser.
- **Inkomst är starkast predictor:** Korrelationsmatrisen visar att median_income har den starkaste korrelationen med målvariabeln ($r \approx 0.69$). Konstruerade features som bedrooms_per_room visade sig ha starkare samband än råvariablerna de bygger på.

3. Metod (ML-flöde)

Train/test-split

Data delades i 80 % träning och 20 % test med *random_state=42* för reproducerbarhet. Testdatan hölls helt unrörd tills den slutliga utvärderingen – detta är avgörande för att undvika att testresultaten 'läcker in' och ger en falskt optimistisk bild av prestandan.

Preprocessing (pipeline)

All preprocessing sker i en scikit-learn Pipeline: medianimputation för saknade värden, StandardScaler för numeriska features och OneHotEncoder för den kategoriska variabeln *ocean_proximity*. Pipelinen anpassas (*fit*) enbart på träningsdata och appliceras (*transform*) på testdata – detta förhindrar data leakage.

Feature engineering

Tre nya variabler konstruerades: *rooms_per_household*, *bedrooms_per_room* och *opulation_per_household*. Dessa normaliserar råsummorna per område och fångar husens karaktär bättre än absoluta totaler.

Modeller och utvärdering

Tre modeller jämfördes med 10-fold cross-validation på träningsdata. **RMSE** (Root Mean Squared Error) valdes som metric eftersom det straffar stora fel hårdare – en stor felvärdering kan leda till dåliga investeringsbeslut. Den bästa modellen optimerades sedan med GridSearchCV (5-fold, 90 kombinationer).

4. Resultat

Jämförelse av modeller (CV på träningsdata)

Modell	CV RMSE (medel)	Standardavvikelse
Dummy Baseline (medelvärde)	115 613 kr	1610 kr
Linear Regression	67 828 kr	2468 kr
Random Forest (standard)	49 683 kr	1913 kr

Hyperparameter-tuning (GridSearchCV)

Random Forest valdes för tuning baserat på lägst CV RMSE. GridSearchCV testade kombinationer av *n_estimators* [3, 10, 30, 100] och *max_features* [2, 3, 4, 6, 8] samt bootstrap=False-varianter. Bästa parametrar: **6 st max_features och 100 n_estimators**

Slutlig test-prestanda

Modell	Test RMSE
Random Forest (tunad)	48 138 kr

5. Rekommendation

Den tunede Random Forest rekommenderas för vidare användning. Modellen presterar betydligt bättre än Linear Regression och vida bättre än Dummy-baslinen. Med ett RMSE på ca 48 000 dollar innebär det att modellens typiska fel är i den storleksordningen – rimligt för grova investeringsbedömningar på områdesnivå.

Random Forest hanterar icke-linjära samband och interaktioner mellan variabler utan manuell konfiguration, vilket gör den robust för detta dataset där samband som pris/geografi är komplexa.

6. Risker & begränsningar

- **Pristaket:** Värden över 500 000 dollar är avklippta i datan. Modellen kan systematiskt underskatta de dyraste områdena.
- **Generalisering:** Datan är från ett specifikt historiskt urval av Kalifornien. Modellen bör inte användas direkt på andra delstater eller nutida data utan omskolning.
- **Aggregerad data:** Alla variabler är summor eller medianer per område, inte per enskilt hus. Individuella husvärderingar kan därför inte göras med denna modell.
- **Data leakage:** Om preprocessing inte görs korrekt (t.ex. om scaler anpassas på hela datasetet) riskerar man att testresultaten är optimistiska. I detta arbete hanterades det med Pipeline.
- **Tolkningssvårighet:** Random Forest är svår att förklara för icke-tekniska beslutsfattare. Feature importance-värden kan ge viss vägledning men ger inte kausala förklaringar.

7. Oövervakad inlärning

Vad är oövervakad inlärning? Oövervakad inlärning är en typ av maskininlärning där modellen tränas på data utan tillhörande svar (labels/targets). Istället för att förutsäga ett känt utfall letar algoritmen efter mönster, strukturer eller grupperingar i datan på egen hand.

När är det relevant i detta case? Ledningen vill förstå om det finns naturliga grupper av områden i Kalifornien – t.ex. 'kustnära lyxområden', 'inlandssuburbia' eller 'tätbebyggda storstadsområden'. Oövervakad inlärning kan hjälpa till att segmentera marknaden och ge beslutsfattare en enklare överblick innan djupare analys.

PCA (Principal Component Analysis)

PCA minskar antalet dimensioner i datan genom att hitta de riktningar (komponenter) som förklarar mest varians. Används för att visualisera högdimensionell data i 2D/3D, identifiera vilka variabler som 'hänger ihop', och ta bort brus inför vidare modellering.

KMeans-klustring

KMeans delar in datapunkter i K grupper där varje punkt tilldelas den kluster vars centrum den är närmast. Används för att segmentera områden i grupper med liknande egenskaper – t.ex. för att hitta 'typområden' som kan användas som beslutsunderlag.

Risker och fallgropar

- **Känslighet för skalning:** Både PCA och KMeans är känsliga för att variabler är på olika skalor. Om man inte standardiserar data first riskerar variabler med stora absoluta värden (t.ex. total_rooms) att dominera analysen helt.
- **Val av K är godtyckligt:** KMeans kräver att man bestämmer antalet kluster i förväg. Det finns ingen 'rätt' siffra – elbow-metoden ger vägledning men det slutliga valet innehåller alltid ett subjektivt inslag. Fel K ger missledande segmentering.

8. Självreflektion

Vad gick bra?

Jag lyckades bygga ett komplett och reproducerbart ML-flöde med korrekt pipeline som förhindrar data leakage. Feature engineering med de tre konstruerade variablerna förbättrade modellens prestanda mätbart. Cross-validation användes konsekvent för att jämföra modellerna på ett rättvist sätt.

Vad var svårast?

Det svåraste var att strukturera notebooken pedagogiskt – att balansera teknisk korrekthet med tydlighet för en icke-teknisk läsare. Att förstå skillnaden mellan att utvärdera på träningsdata vs testdata vs cross-validation tog tid att sätta sig in i ordentligt.

Betygssjälvskattning: G

Jag bedömer att inlämningen motsvarar betyget **G**. Följande G-krav är uppfyllda:

- EDA med minst 2 relevanta figurer (histogram + geografisk scatter) med skriftliga tolkningar.
- Korrekt train/test-split (80/20, random_state=42) utan leakage via Pipeline.
- Minst 3 modeller jämförda med samma metric (10-fold CV RMSE): Dummy, Linear Regression, RandomForest.
- Hyperparameter-tuning med GridSearchCV på vald modell, optimerad mot neg_mean_squared_error.

- Slutlig utvärdering på osedd testdata med motiverad metric (RMSE).
- Tydlig rekommendation kopplad till metric och affärssyfte.

VG-kraven (oövervakad implementation + 2 fördjupningar) är *inte* implementerade i koden, varför VG inte kan motiveras.